TECHNICAL ASSESSMENT · DESIGN DOCUMENT

Gatekeeper

Nagarkot Forwarders Pvt. Ltd.
November 2026

Access control for NFPL customs jobs: who may do what, on which job, and why Node.js · TypeScript · Express · Prisma · PostgreSQL

The problem. A filed shipment was altered and nobody could say **who had been allowed to, or why**. The design treats that as three requirements, and keeps the system easy to run day to day while meeting them.

REFUSE what shouldn't happen, by default

EXPLAIN every yes and every no, in a sentence

REMEMBER enough to answer questions about the past

1 The model

Job the thing being protected

Four **access attributes**: client, branch, direction (IMPORT/EXPORT), mode (AIR/FCL/LCL). They are fixed after creation, because they decide who may touch the job. It also has a stage and a version number.

CONFIRMED → PREPARED → CHECKED → FILED 🔒

plus CANCELLED and REOPENED

AccessGrant the core idea

```
user × role × scope × [from, until)
```

A **scope** is any mix of client, branch, direction and mode, where an empty field means "any". A person holds as many grants as they need. Grants are **never deleted**: a revoke records who, when and why, so the past stays answerable.

Role & permission stored as data

Permissions are fine-grained verbs (`job.check`, `job.cancel_filed`, `access.manage`, …). A role is a named bundle stored as a row: MAKER, CHECKER, FILER, VIEWER, CLIENT_VIEWER, OPS_ADMIN, ACCESS_ADMIN, SYSTEM_PARSER, SYSTEM_AUTOMATION.

Delegation temporary cover

"A's role R for scope S is exercised by B until T." B acts in their own name, **on behalf of** A, and history records both.

Absence & status

An **absence** blocks *work* (not viewing) for a period. Status **INACTIVE** blocks everything, including existing tokens.

Workflow rules also data

Which stage each action may start from, which actions need a written reason, and separation-of-duties rules ("you may not check a job you prepared").

The decision engine: `decide(context, action, job, history, at)` pure functions with no database access · stops at the first failed check · every check adds a line to the trace

1	2	3	4	5	6	7	8	9
identity & active	known action	job exists	stage allows it (filed lock)	not absent	client boundary	role has permission	grant or delegation scope covers the job	separation of duties

ALLOW DENY

plus a code (`SCOPE_MISMATCH`, `SOD_VIOLATION`…), a plain sentence, the full trace, and the exact grant or delegation that matched. The **same function** powers enforcement, explanations, history replays, queues and routing, so there is only one copy of the rules.

JobHistory append-only

Every change: actor, on-behalf-of, the **grant or delegation that authorised it**, stage before and after, reason, field-level diff.

AccessAuditLog append-only

Every access change, **and every denied attempt** (for example an admin trying to grant themselves a role).

AutomationLog append-only

Everything the router and the overnight parser did, or failed to do, with the reason.

2 Key decisions and trade-offs

DECISION	WHY	TRADE-OFF ACCEPTED
Scoped grants (role × scope), not one role per user or roles like `CHECKER_ACME_BOM`	Matches how NFPL works: people cover slices of the business. No role explosion.	More complex queries, solved by compiling scopes into SQL filters.
One pure engine used everywhere	Lists, queues, routing and explanations can't drift from enforcement. Unit-tested without a database (33 tests).	Callers must load the facts first.
Token carries only the user id	Access is read from the DB on each request, so a revoke or deactivation applies on the next click, not when the token expires.	One small DB read per request; a short cache could come later.
Deny by default, fail closed	Unknown action, missing data or any exception becomes `DENY / INDETERMINATE`.	A bug causes a refusal, never a silent allow.
404 when you may not even see a job	Doesn't reveal which jobs exist, especially to client users.	Callers can't tell "missing" from "hidden"; admins use the explain API.
Optimistic concurrency (version column)	Two simultaneous "check" clicks: exactly one wins, the other gets 409. No locks held.	The loser must reload.
Synchronous routing after each change	Simple and testable; enough for 120 staff.	Under heavy load, move to an outbox + worker.
Privileged roles are bootstrap-only	Admins can't create more admins through the API, not even for each other.	Adding an admin is a deliberate seed/migration step.

Lists stay fast at 200,000 jobs. Visibility isn't checked job by job: a user's grants compile into one SQL `WHERE` (an OR of scopes, each an AND of the fixed attributes), and the database filters and paginates using composite indexes on the job attributes, assignee and history. Measured once on a laptop with 500 users and 200,008 jobs:

15–35 ms	16–62 ms	8 ms	44 ms
job list page	jobs waiting for me	job history	who-could-act, all 500 users

3 Answers to the questions

1 Should everyone see everything?

No. Viewing is a permission like any other (`job.view`), scoped the same way: a checker for ACME/BOM sees ACME/BOM jobs. **Seeing is wider than acting**: a MAKER for all clients sees everything but can only prepare, and a VIEWER sees but can't act. Client users are also clamped to their own company whatever their grant says, and their history view shows "NFPL" instead of staff names. Jobs you can't see return 404, not 403.

2 Maker everywhere, Checker for a few? Checker for one client, branch, imports only?

Several grants per person, each with its own scope. **Sarah:** `MAKER (any)` + `CHECKER (ACME)` + `CHECKER (BLUESKY)`. **Raj:** one grant, `CHECKER (ACME, BOM, IMPORT, any mode)`. The system describes this in plain English (`GET /auth/me`, `npm run access:describe`).

3 The preparer is also qualified to approve it?

Qualification isn't enough. A separation-of-duties rule runs *after* the scope check: if you prepared this job (per its history), you can't check or reject it. Sarah can check ACME jobs, just not her own. Routing and "waiting for me" apply the same rule, so she is never even offered it. The same idea stops an admin who checked or filed a job from cancelling or reopening it.

4 The person responsible is absent?

An **absence** blocks work actions (viewing still works). Recording one immediately re-checks that person's open jobs and re-routes each to someone the engine says can do the next step. If nobody can, the job is left unassigned and flagged in the automation log for a human to handle. Cover is explicit: a **delegation** lets Dev do Priya's CORAL checking until a set date, and history records "Dev, on behalf of Priya, under delegation X". A delegation *lends scope, not qualification* (Dev must already be a CHECKER). It reaches only what Priya holds *at the moment of use*, is capped at 30 days, and an admin can't set up one they are part of.

5 Jobs halfway through when responsibilities change?

Authority: a revoke takes effect on the next request. Work already done stays valid, and its history still points to the grant that allowed it at the time. **Assignments:** revoking a grant, ending a delegation or deactivating a user re-validates that person's open jobs and re-routes the ones they can no longer do. The job keeps its stage; only the next actor changes.

6 Who can cancel a filed job or reopen a closed one?

Filed jobs are **locked for everyone** by the stage check. Cancelling a filed job is a separate, more sensitive permission (`job.cancel_filed`) from cancelling a draft. It and `job.reopen` belong only to OPS_ADMIN, **scoped** (Olivia: DEL only), need a written reason, and are blocked for an admin who checked or filed that job. Each is recorded with the reason and the grant used. *Assumption:* one in-scope admin plus a reason is enough. Two-person approval was left out as too heavy day to day, but it would slot in as one more check.

7 A client's employee follows their shipments?

A `CLIENT` user type with a `CLIENT_VIEWER` grant for their own client. Three safeguards: the role only has `job.view`; the engine clamps client users to their own company even if a grant were misconfigured; and admins can only give client users read-only roles for their own client. They see the status trail without internal names.

8 An admin fixes a stuck job, or tries to gain power?

Fixing: OPS_ADMIN can reassign with a reason, but only to someone the engine says can do the next step. They can't do the work for someone else or force a stage (there is no "set stage" endpoint). **Escalation:** any access change where actor = target is refused (`SELF_ESCALATION`), privileged roles can't be granted at all, and changes outside the admin's own scope are refused (Bharat is DEL-only). Every attempt, allowed or denied, is audited.

9 The overnight parser?

As a user, not a backdoor. It is a `SERVICE` identity with no password, whose SYSTEM_PARSER role allows only creating jobs and updating imported fields, and only before preparation. It goes through the same services and engine as people, so a filed job rejects its update the same way. Re-runs are safe: records match on `externalRef` and unchanged data writes nothing. It may not change client, branch, direction or mode, since that would silently move a job into other people's scope. Bad records are logged and skipped; the batch continues.

10 "Why can she do this and he can't?"

`POST /authorization/check` with several user ids runs the engine for each and returns, side by side, the decision, a sentence and the trace:

> *Sarah: has CHECKER for client ACME … which covers this EXPORT AIR job for ACME at DEL.*
> *Raj: no CHECKER access covers this job. Closest: CHECKER for ACME, BOM, IMPORT; mismatch on branch (job is DEL), direction (job is EXPORT).*

The "closest grant" hint tells an admin exactly what to change.

11 "Who could have changed this job on the 3rd?"

It's three questions. **Who did** comes from JobHistory. **Under what authority** comes from the grant or delegation id stored on each history row. **Who could have** comes from `GET /jobs/:id/who-could-act?at=…`, which rebuilds the facts as they were at that moment (the job's stage from its history, grants and delegations by their validity and revoke dates, absences) and re-runs the same engine for every user. Nothing is overwritten, so this works for any past time.

12 A new kind of role next quarter, engineering busy?

Roles are data: a new role is one row (name + permission list), granted through the normal scoped-grant API, with no code change or deploy. Engineering is only needed for a genuinely new *verb* (permission) or workflow rule. An admin screen for creating roles was cut for time; today it's a one-line seed or SQL change.

13 "Waiting for me" matches what I can actually do?

It isn't a stored task list that could go stale. It's a query built from the same parts as the engine (my grants' scopes for the job's next action, the separation-of-duties rule, assigned to me or unassigned), computed fresh on each request. A test walks every user's queue and confirms the engine says ALLOW for each item, so any drift fails the build.

14 When the system is unsure?

It says no, and says why. Unknown actions, missing facts or any exception during evaluation become `DENY / INDETERMINATE`. Routing that can't find a safely eligible person leaves the job unassigned and flags it rather than guessing. Ambiguous input (missing reason, bad dates, a parser record that would move scope) is rejected, not repaired.

+15 Two people act on the same job at once?

Optimistic concurrency: exactly one wins, the other gets 409. A test fires two simultaneous checks and asserts there is one CHECK in history.

+16 How fast does a revoke hit someone already logged in?

On their next request, because tokens hold identity only. A deactivated user's token stops working at once.

+17 Can a client user learn other clients' jobs exist?

No. Out-of-scope jobs return 404 and lists are filtered in SQL.

4 Assumptions and what was cut

Assumptions (where the brief is silent)

- One NFPL organisation.
- A job's client, branch, direction and mode don't change after creation.
- "Reopen" returns a job to a stage where it can be prepared again.
- A written reason of at least 3 characters is enough.
- Absence cover is short (a delegation of 30 days at most); longer changes are new grants.

Cut for time (not claimed as built)

- Configurable automation rules and an event bus.
- Overdue-job escalation and notifications (both depend on that rule engine).
- An admin API/UI for creating roles; a parser HTTP endpoint; a UI.
- **Next, in order:** outbox + worker for routing → escalation of stale jobs → role-management API → a short-lived permission cache if request volume grows.